

Course

« *Computer Vision* »

Sid-Ahmed Berrani

2025-2026



IV. Edge Detection: The Canny Detector

- Probably the most widely used edge detector in computer vision.
- It uses the best properties of the gradient operator and the Laplacian.
- In 1986, Canny tried to find the **optimal** edge detector, assuming a perfect step edge in Gaussian noise. Optimal means:
 1. **Low error rate:** All edges should be found, and there should be no spurious responses.
 2. **Edge points should be well localized:** The edges located must be as close as possible to the true edges
 3. **Single edge point response:** The detector should not identify multiple edge pixels where only a single edge point exists.

IV. Edge Detection: The Canny Detector

- The essence of Canny's work was in expressing the preceding three criteria mathematically and then attempting to find optimal solutions to these formulations.
- Given a filter F , two objective functions have been defined:
 1. $\Lambda(F)$: Large if F produces good **localization**.
 2. $\Sigma(F)$: Large if F produces good **detection**.

Problem:

Find the best filter F that maximizes the compromise criterion: $\Lambda(F) \Sigma(F)$.

With the additional constraint that a single peak should be generated at a step-edge.

IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- The algorithm:
 - Smooth the image with a 2D Gaussian filter.
 - Compute Image gradient using Sobel operator (for example).
 - Find gradient magnitude at each pixel.
 - Find gradient orientation at each pixel.
 - Compute 1D Laplacian along the gradient direction at each pixel.
 - Find zero-crossings in Laplacian to find the edge location.

IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- Smooth the image with a 2D Gaussian filter:

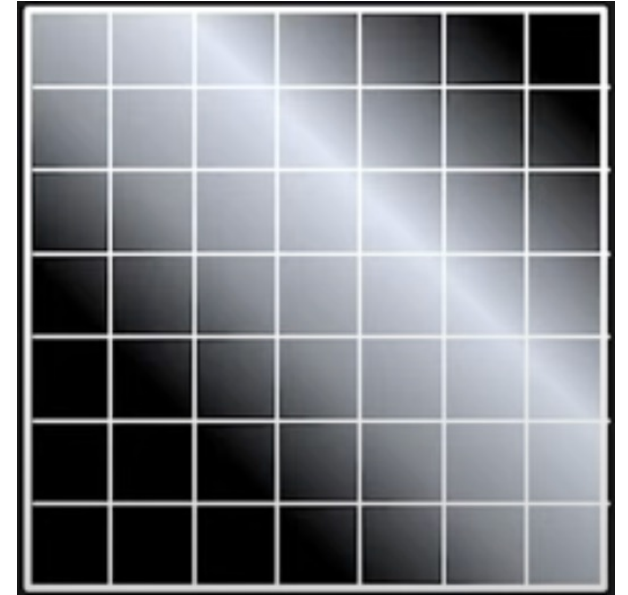
$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$



IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

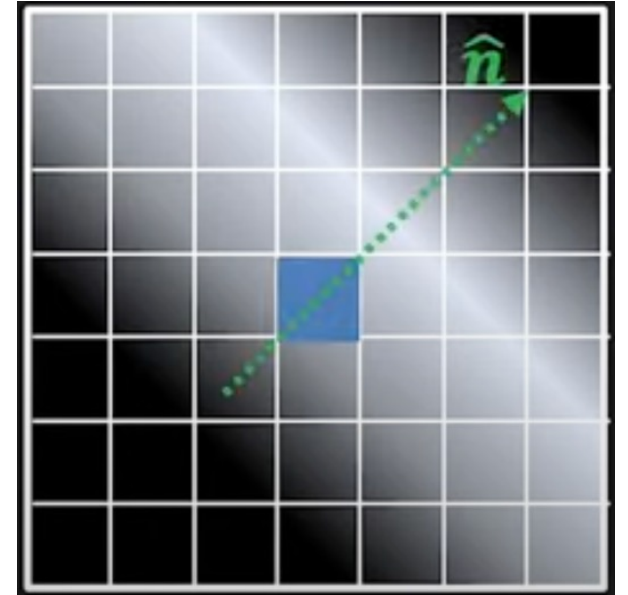
$$\nabla n_{\sigma} * I$$

- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$

- Find gradient direction at each pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$



IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

- Find gradient magnitude at each pixel:

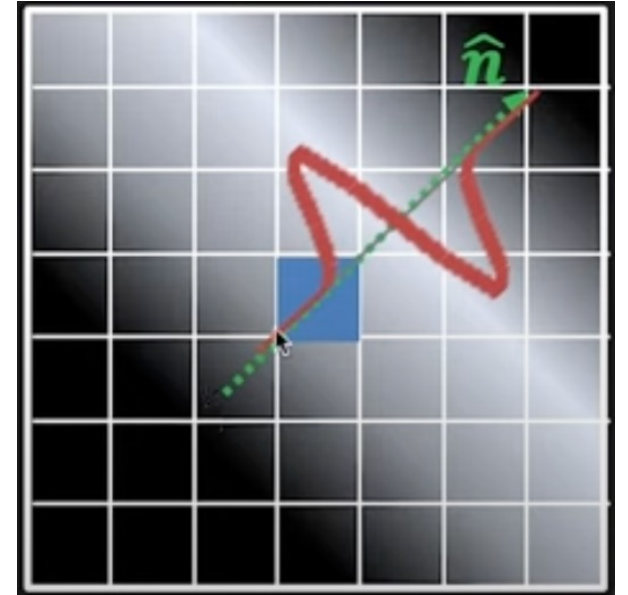
$$\|\nabla n_{\sigma} * I\|$$

- Find gradient direction at each pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Compute 1D Laplacian along the gradient direction $\hat{n}$ at each pixel:

$$\frac{\partial^2 (n_{\sigma} * I)}{\partial \hat{n}^2}$$



IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$

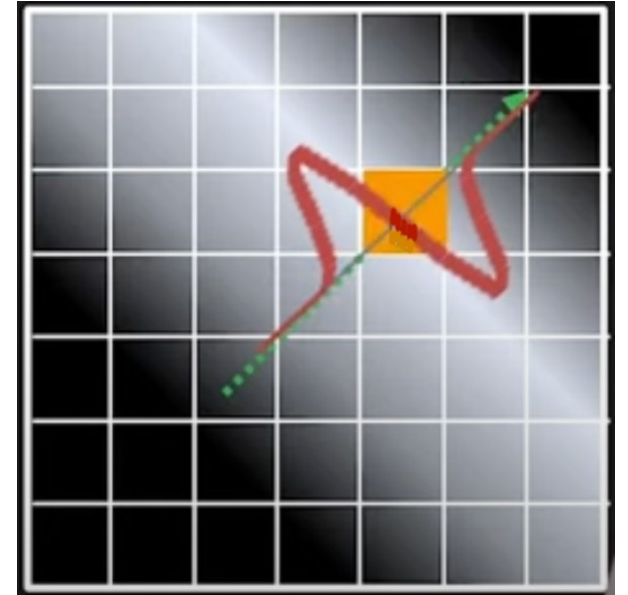
- Find gradient direction at each pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Compute 1D Laplacian along the gradient direction $\hat{n}$ at each pixel:

$$\frac{\partial^2 (n_{\sigma} * I)}{\partial \hat{n}^2}$$

- Find zero-crossings in Laplacian to find the edge location.



IV. Edge Detection: One way to make use of both the gradient and the Laplacian

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$

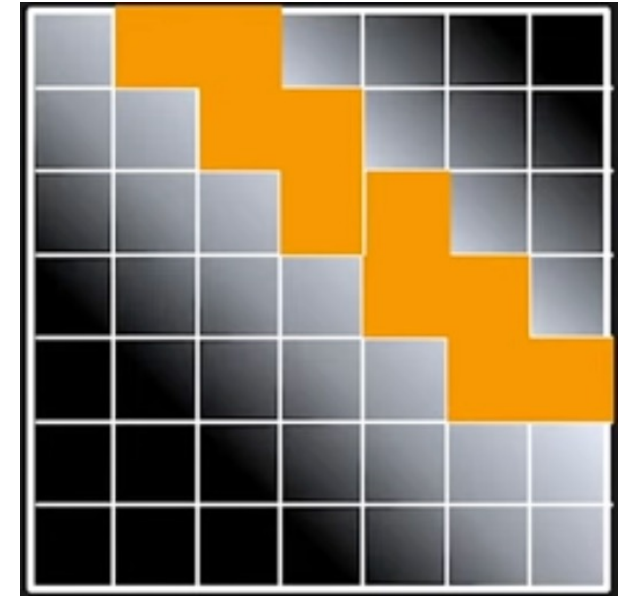
- Find gradient direction at each pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Compute 1D Laplacian along the gradient direction $\hat{n}$ at each pixel:

$$\frac{\partial^2 (n_{\sigma} * I)}{\partial \hat{n}^2}$$

- Find zero-crossings in Laplacian to find the edge location.



IV. Edge Detection: The Canny Detector

- The algorithm:
 - Smooth the image with a 2D Gaussian filter.
 - Compute Image gradient using Sobel operator (for example).
 - Find gradient magnitude at each pixel.
 - Find gradient orientation at each pixel.
 - Apply a non-maxima suppression approach (the idea is to select the single maximum point across the width of an edge).
 - Threshold the non-maxima suppressed gradient image to reduce false edge points – Hysteresis thresholding is used here.

IV. Edge Detection: The Canny Detector

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

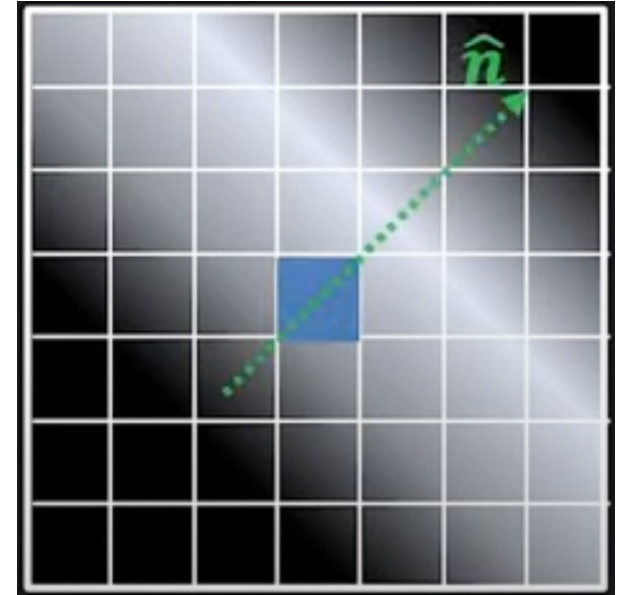
- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$

- Find gradient direction at each pixel:

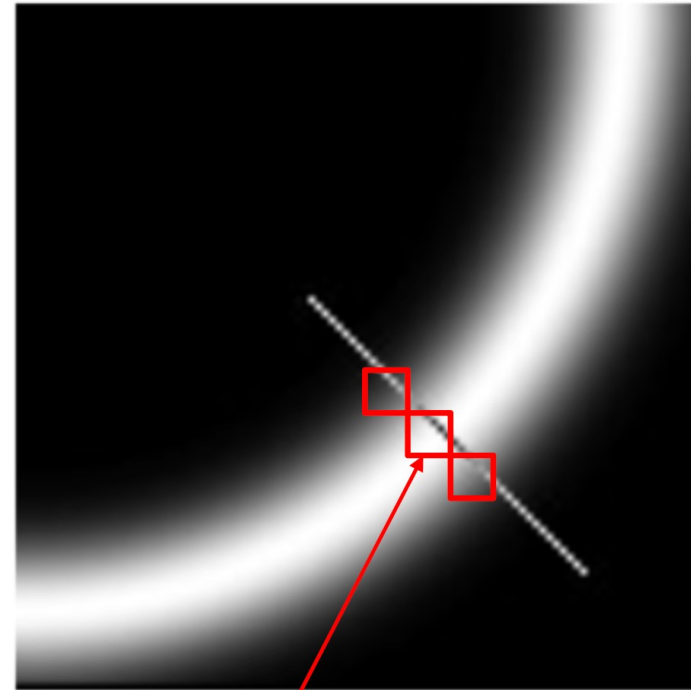
$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Apply the non-maxima suppression: For each pixel check the neighbors depending on the direction of the gradient



IV. Edge Detection: The Canny Detector

- The non-maxima suppression



If the intensity of the gradient of a pixel is less than at least one of the two neighbors along the gradient direction, then it must be removed (i.e. gradient magnitude set to zero)

IV. Edge Detection: The Canny Detector

- Smooth the image with a 2D Gaussian filter:

$$n_{\sigma} * I$$

- Compute Image gradient using Sobel operator (for example):

$$\nabla n_{\sigma} * I$$

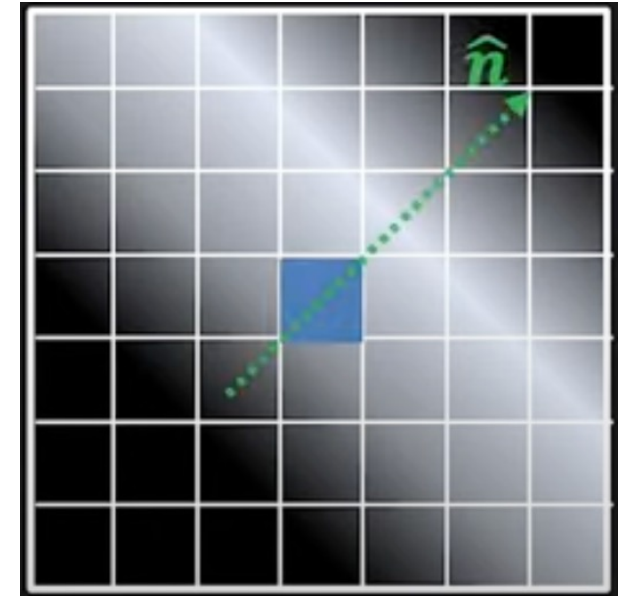
- Find gradient magnitude at each pixel:

$$\|\nabla n_{\sigma} * I\|$$

- Find gradient direction at each pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Apply the non-maxima suppression: For each pixel check the neighbors depending on the direction of the gradient.
- Apply hysteresis thresholding to reduce false edge points.



IV. Edge Detection: The Canny Detector

Thresholding:

- Hysteresis-based: Using two thresholds ($T_0 < T_1$)

$$\|\nabla I(x, y)\| < T_0$$

Definitely not an edge

$$\|\nabla I(x, y)\| \geq T_1$$

Definitely an edge

$$T_0 \leq \|\nabla I(x, y)\| < T_1$$

Is an edge if a neighboring pixel is definitely an edge

IV. Edge Detection: The Canny Detector

Step 1: Noise reduction with Gaussian blur – smooth the image using a Gaussian filter.

Step 2: Compute gradient magnitude and direction.

Step 3: Keep only the local maxima in the gradient direction – this step thins the edges. Can be presented as “**a non-maxima suppression**” or “**a 1D Laplacian operator**”.

Step 4: Apply hysteresis thresholding to reduce false edge points.

IV. Edge Detection: The Canny Detector

- An example: The Lena image



σ_1

$$\sigma_1 = 1$$



σ_2

$$\sigma_2 = 2$$



σ_3

$$\sigma_3 = 4$$

$\sigma_1 ? \sigma_2 ? \sigma_3$

$\sigma_1 > \sigma_2 > \sigma_3 ?$

$\sigma_1 < \sigma_2 < \sigma_3 ?$

IV. Edge Detection: The Canny Detector

- Another example



Sigma= 1, Low= 0.4, High= 0.8



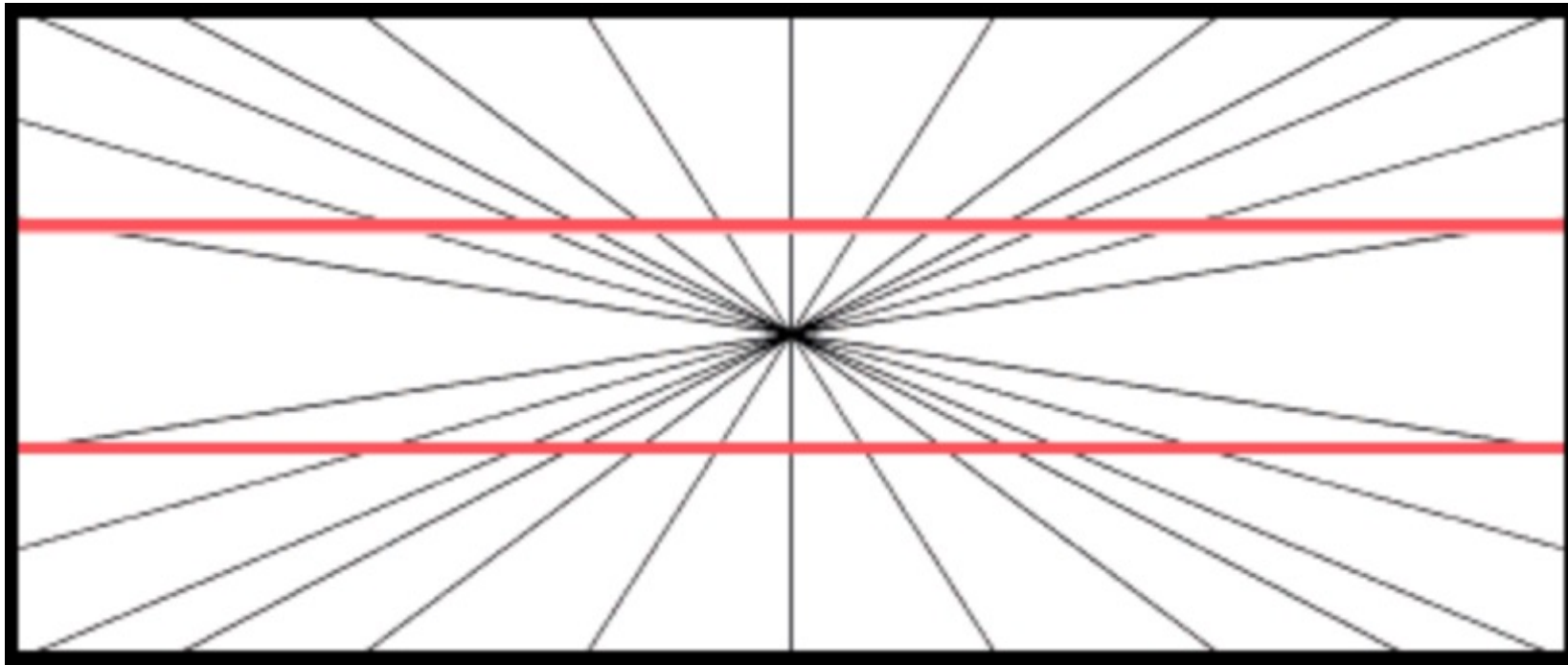
Sigma= 2, Low= 0.4, High= 0.8



Sigma= 1, Low= 0.3, High= 0.7

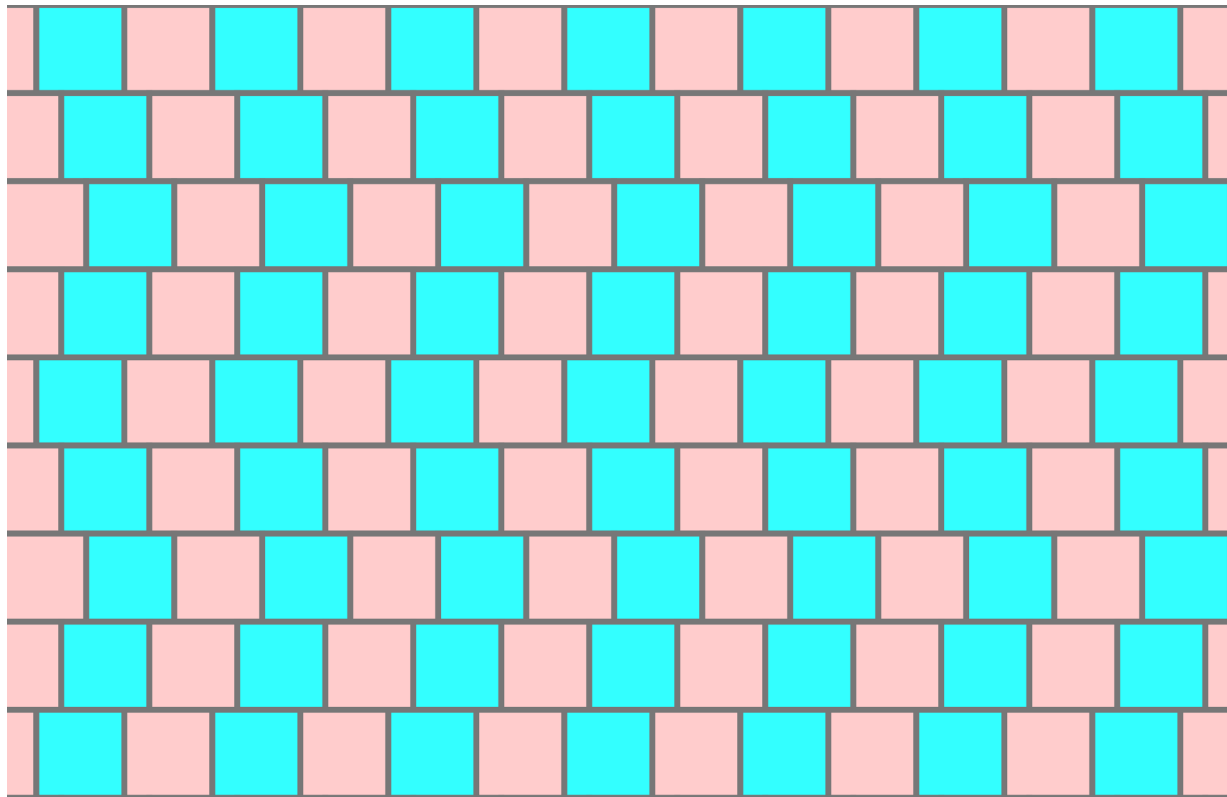
V. Edge Detection: Few Optical Illusions

- The standard Hering illusion (1861).



V. Edge Detection: Few Optical Illusions

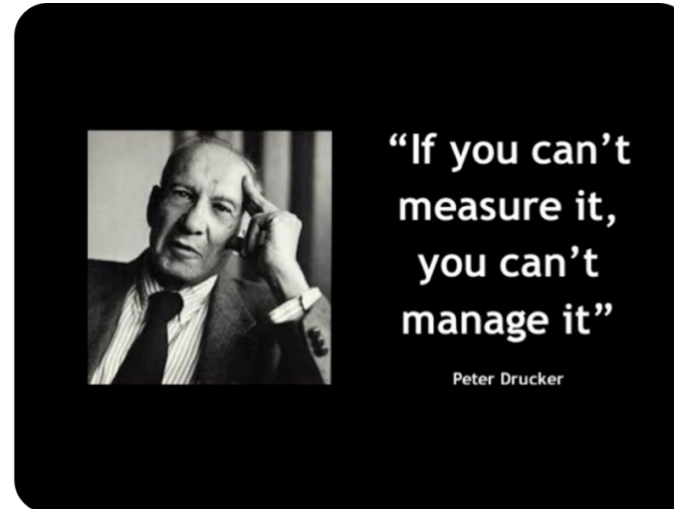
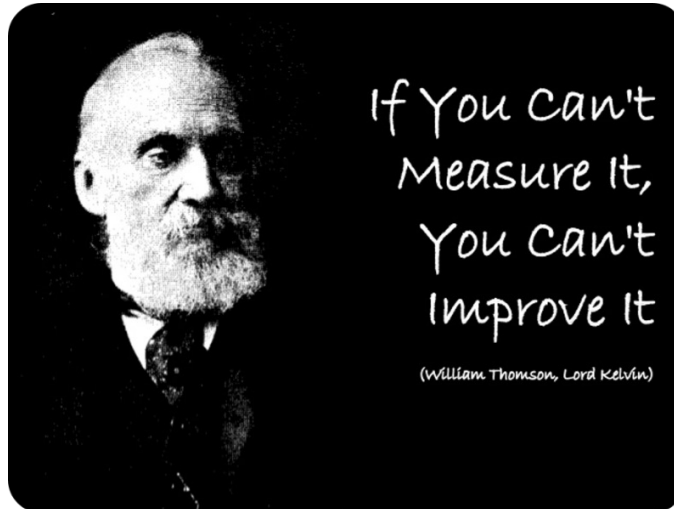
- The Café wall illusion (1979).



V. Edge Detection Evaluation

Why evaluate edge detectors?

- Compare performance of different methods.
- Choose the most suitable detector for a target application.
- Ensure reliability in real-world tasks.



V. Edge Detection Evaluation

The 1st approach: Qualitative evaluation

⇒ Based on visual inspection:

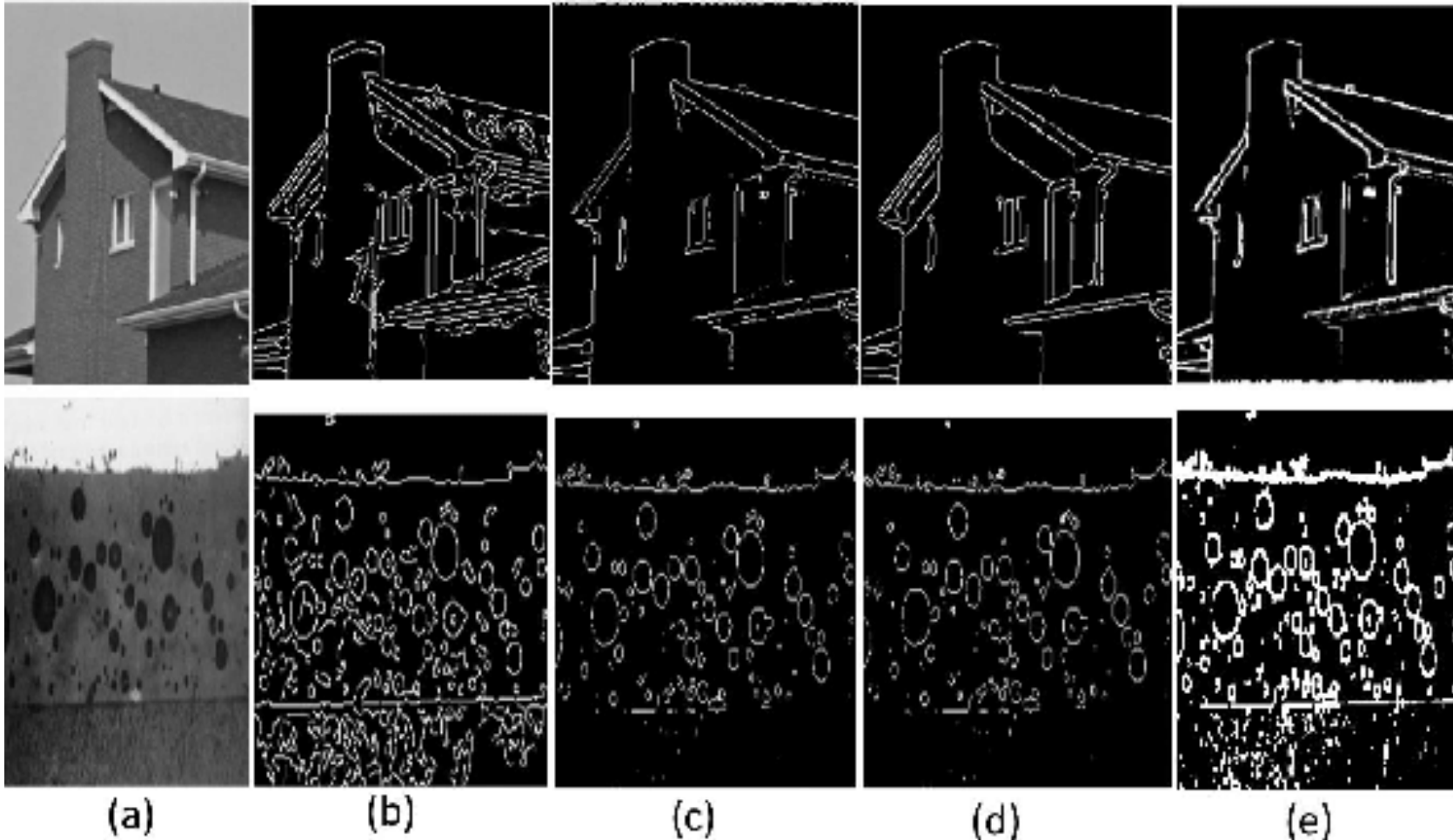
- Are the edges **clean, thin, and well-localized**?
- Are object **boundaries complete** and continuous?
- Is there too much **noise** (false edges)?
- Do the results match **human perception**?

Common practice: overlay edge map on original image.

⇒ **Quick, intuitive, but subjective.**

V. Edge Detection Evaluation

The 1st approach: Qualitative evaluation



V. Edge Detection Evaluation

The 2nd approach: Quantitative evaluation

- Assumes a ground-truth edge map is available



V. Edge Detection Evaluation

The 2nd approach: Quantitative Evaluation — Key Metrics

- Precision, Recall, F1-score
- ROC Curve & AUC
- **Boundary Displacement Error (BDE)**: Measures the average distance between detected edges and ground truth edges. Lower BDE → better alignment with true boundaries.
- **Pratt's Figure of Merit (FOM)**: see next slide.
- **Edge Localization Error**: Measures how precisely edges are detected in terms of position, especially important in high-resolution images or critical applications.

⇒ **Objective, repeatable, and statistically measurable.**

V. Edge Detection Evaluation

The 2nd approach: Quantitative Evaluation — Key Metrics

- Pratt's Figure of Merit (FOM)

$$FOM = \frac{1}{\max(N_d, N_g)} \sum_{i=1}^{N_d} \frac{1}{1 + \alpha d_i^2}$$

N_d : number of detected edge pixels

N_g : number of ground truth edge pixels

d_i : distance from each detected edge pixel to the nearest ground truth

α : a constant (typically 1/9)

=> Values close to 1 indicate good performance.

V. Edge Detection Evaluation

In addition, computational metrics can also be used:

- Runtime performance
- Memory usage
- Scalability to large images

⇒ These can be important in real-time or resource-constrained systems.

V. Edge Detection Evaluation

How is Ground-Truth Created?

- Manual annotation by humans (pixel-accurate)
- Multiple annotators: consensus or averaging
- Synthetic images with known edge structure (ideal for testing)

Available datasets with ground-truth:

- BSDS500: Natural images, human-labeled contours
- NYUDv2: RGB-D indoor scenes with edges
- PASCAL Boundaries: Object-level boundaries
- SBD: Semantic boundaries from PASCAL VOC
- BSDS300: Classic version, still cited

V. Edge Detection Evaluation

Best Practices in Evaluation:

- Align image resolutions (ground-truth and prediction)
- Allow small tolerance in edge location
- Avoid data leakage (do not evaluate on training data)
- Combine quantitative and qualitative evaluation

VI. Line/curve boundary detection

Basically:

We have a set of edges in an image $\Rightarrow$ fit a line, a circle or any geometrical shape.

Fitting is the process to decompose an image or a set of tokens (i.e. pixels, isolated points, sets of edge points...) into components that belong to circles, lines or any other shape.

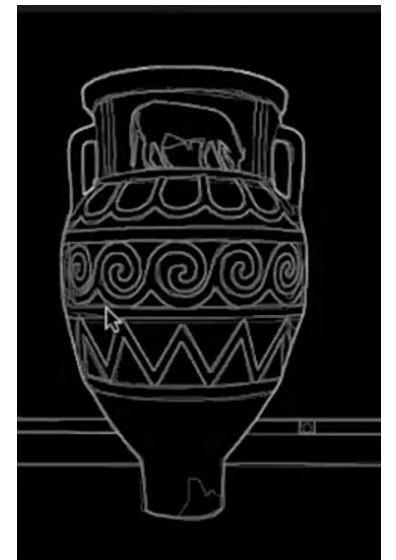
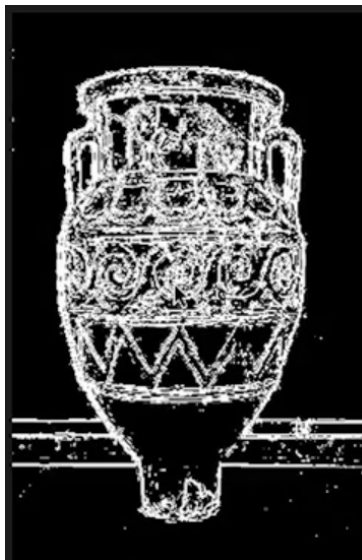
A line



A circle

VI. Line/curve boundary detection

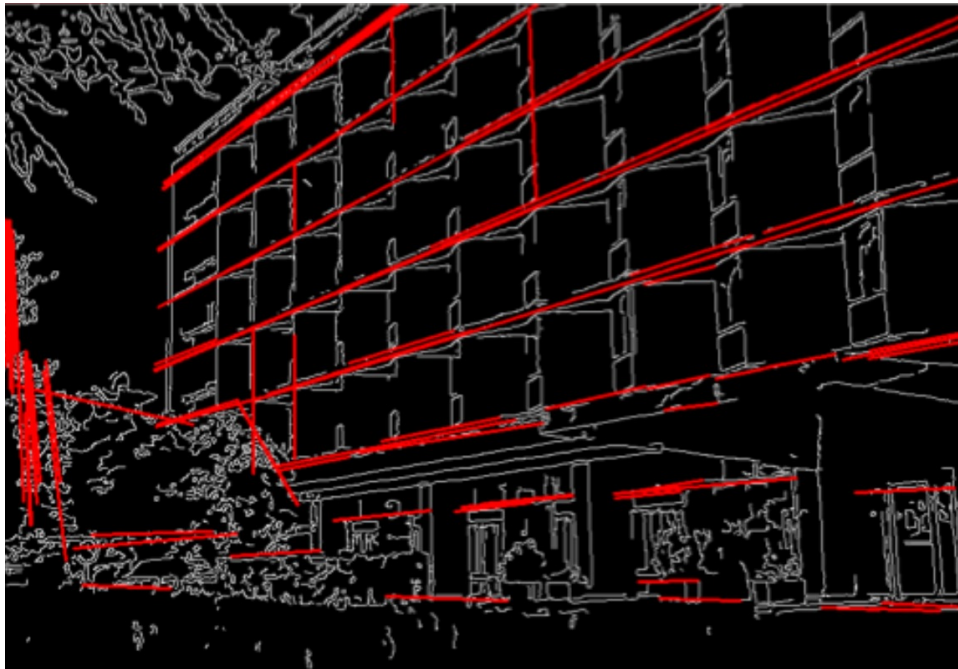
Edge post-processing



VI. Line/curve boundary detection

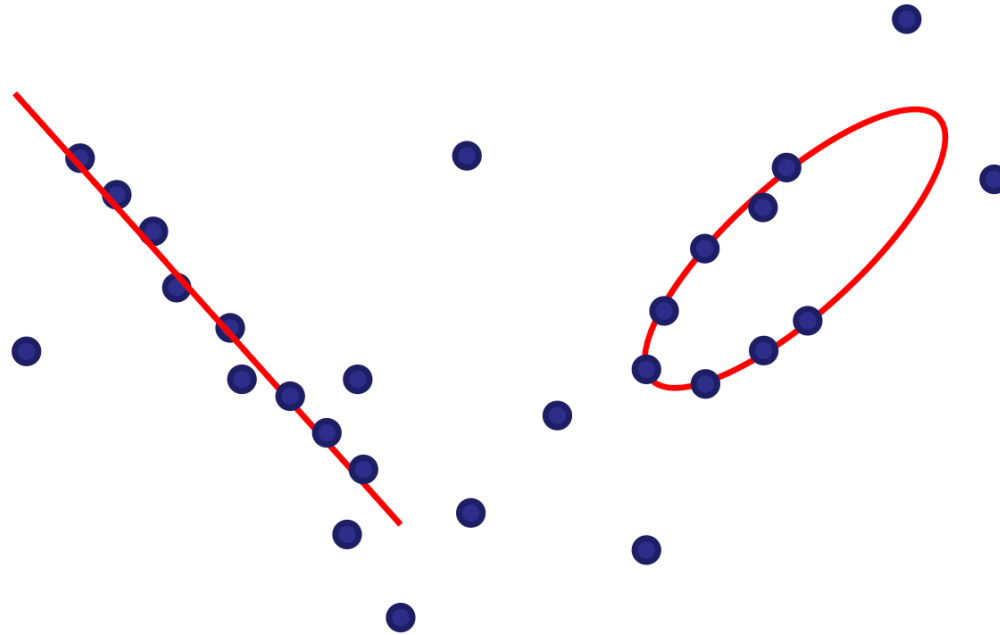
What is it used for?

- Image segmentation: produce compact representations that emphasize the relevant image structures.
- Image understanding.
- Analyzing and measuring man made objects (e.g. as part of quality insurance process).



VI. Line/curve boundary detection

⇒ **Fitting** involves determining what possible curves could have given rise to a set of tokens observed in an image.

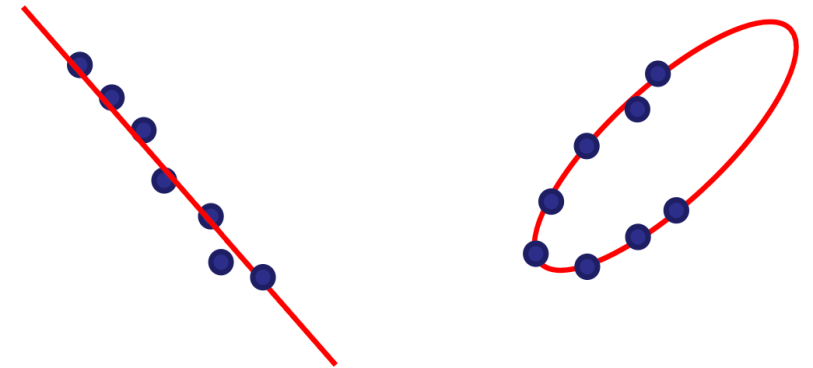


VI. Line/curve boundary detection

Fitting involves determining what possible curves could have given rise to a set of tokens observed in an image.

⇒ **Many sub-problems:**

1. Parameter estimation: if we already know the association between tokens and curves. We need to recover the parameters of each curve.

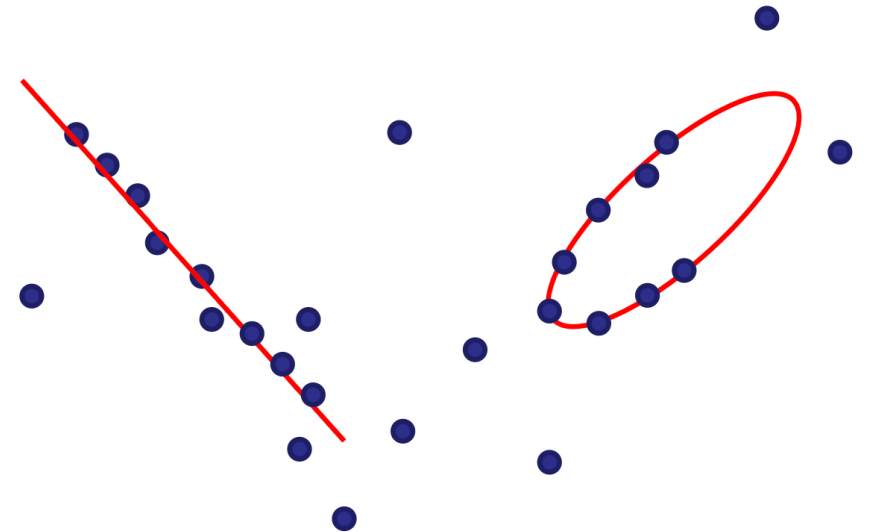


VI. Line/curve boundary detection

Fitting involves determining what possible curves could have given rise to a set of tokens observed in an image.

⇒ **Many sub-problems:**

2. Token-curve association: we assume to know only how many curves are present but not which token came from which curve. The association must be solved together with parameter estimation.



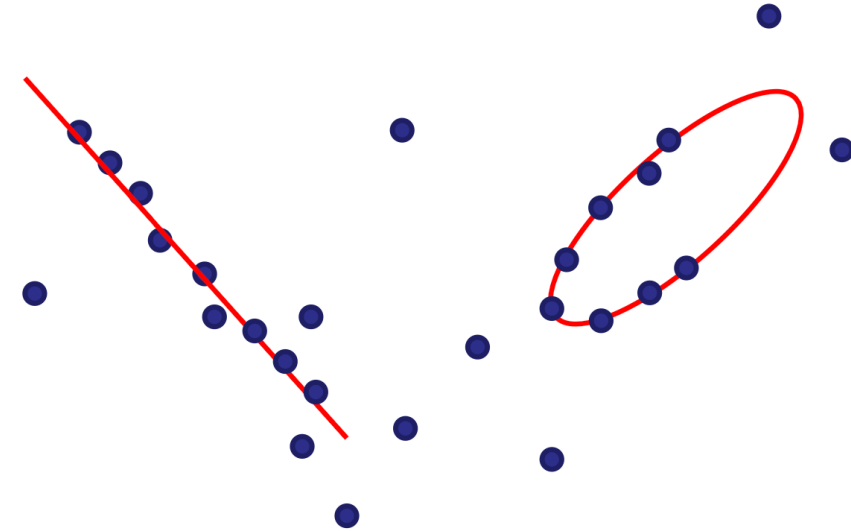
VI. Line/curve boundary detection

Fitting involves determining what possible curves could have given rise to a set of tokens observed in an image.

⇒ **Many sub-problems:**

3. Counting: we have no prior knowledge on the data, so we must figure out:

- (i) How many curves are present,
- (ii) The association between tokens and curves,
- (iii) Curve parameters.



VI. Line/curve boundary detection

Parameter estimation:

- We have observed a set of points generated by a certain curve model with unknown parameters.

Goal: Find the best set of parameters that justify the observations.

Two approaches:

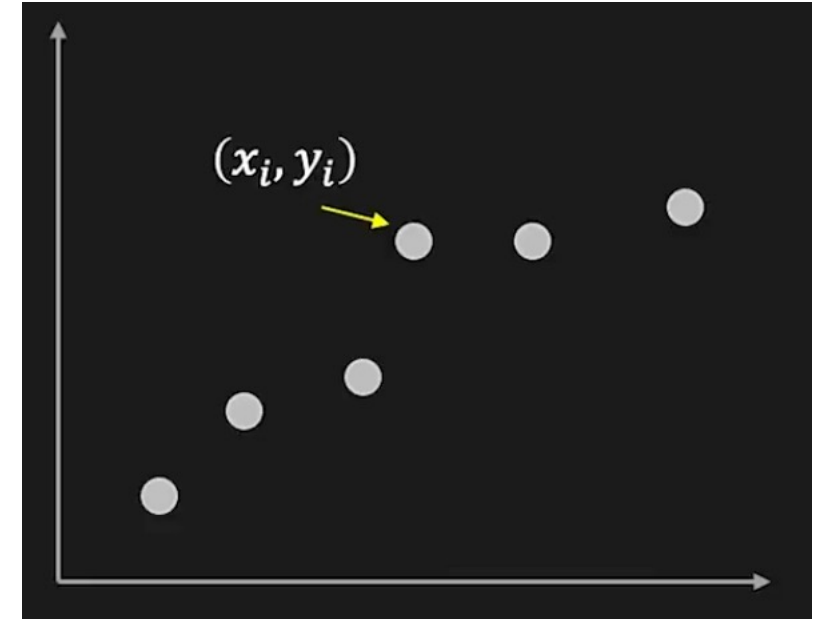
1. Minimize a loss function accounting for the distances between each point and the curve.
2. Describe the curve as a generative model and find the best parameters maximizing the probability of generating the observed data.

In some cases (like 2D lines) the two approaches yield to the same result

VI. Line/curve boundary detection

Let's start with fitting lines to edges...

Edges: points (x_i, y_i)

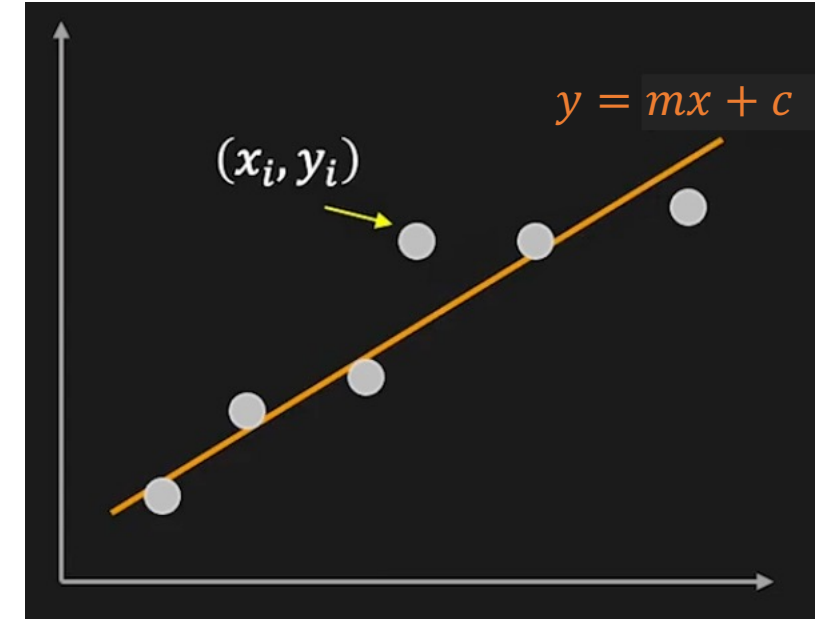


VI. Line/curve boundary detection

Let's start with fitting lines to edges...

Edges: points (x_i, y_i)

Goal: find m and c .



VI. Line/curve boundary detection

Let's start with fitting lines to edges...

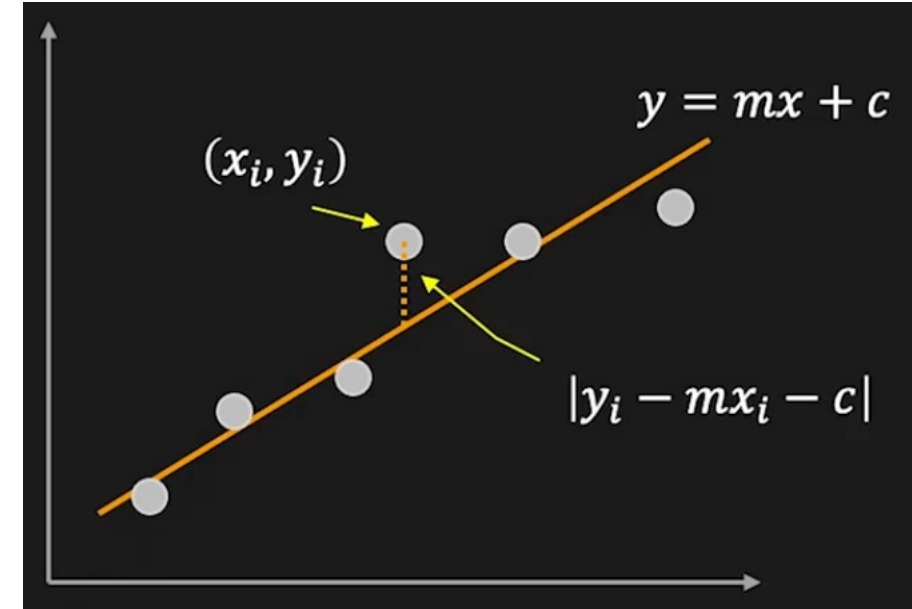
Edges: points (x_i, y_i)

Goal: find m and c .

The idea:

Minimize average squared **vertical** distances.

$$E = \frac{1}{N} \sum_i (y_i - mx_i - c)^2$$



The solution: least square method

$$\frac{\partial E}{\partial m} = \frac{-2}{N} \sum_i x_i (y_i - mx_i - c)$$

$$\frac{\partial E}{\partial c} = \frac{-2}{N} \sum_i (y_i - mx_i - c)$$

VI. Line/curve boundary detection

Let's start with fitting lines to edges...

The idea:

Minimize average squared **vertical** distances.

$$E = \frac{1}{N} \sum_i (y_i - mx_i - c)^2$$

The solution: least square method

$$\frac{\partial E}{\partial m} = \frac{-2}{N} \sum_i x_i (y_i - mx_i - c)$$

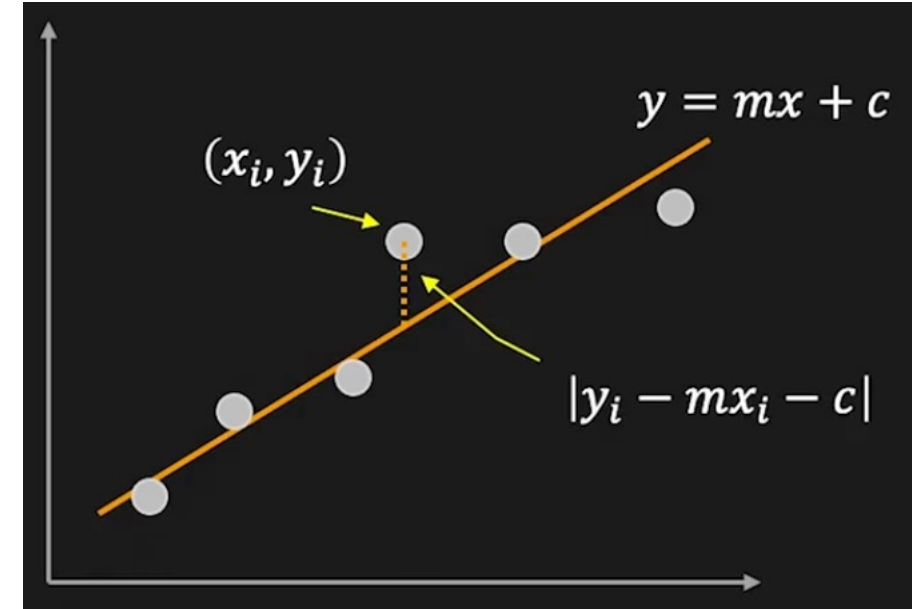
$$\frac{\partial E}{\partial c} = \frac{-2}{N} \sum_i (y_i - mx_i - c)$$

$$m = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

Where

$$\bar{x} = \frac{1}{N} \sum_i x_i \quad \bar{y} = \frac{1}{N} \sum_i y_i$$



VI. Line/curve boundary detection

Let's start with fitting lines to edges...

The idea:

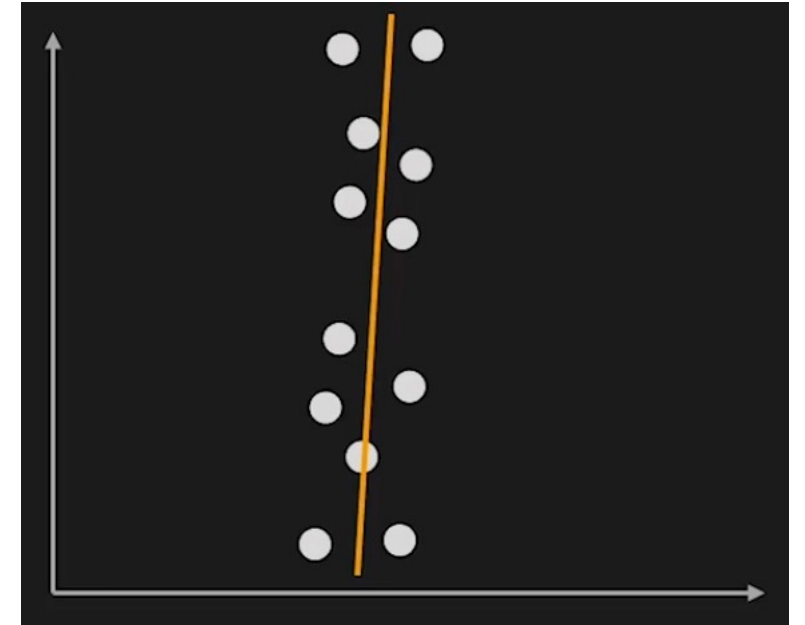
Minimize average squared **vertical** distances.

$$E = \frac{1}{N} \sum_i (y_i - mx_i - c)^2$$

The solution: least square method

The problem:

Vertical lines.



VI. Line/curve boundary detection

Let's start with fitting lines to edges...

The idea:

Minimize average squared **vertical** distances.

$$E = \frac{1}{N} \sum_i (y_i - mx_i - c)^2$$

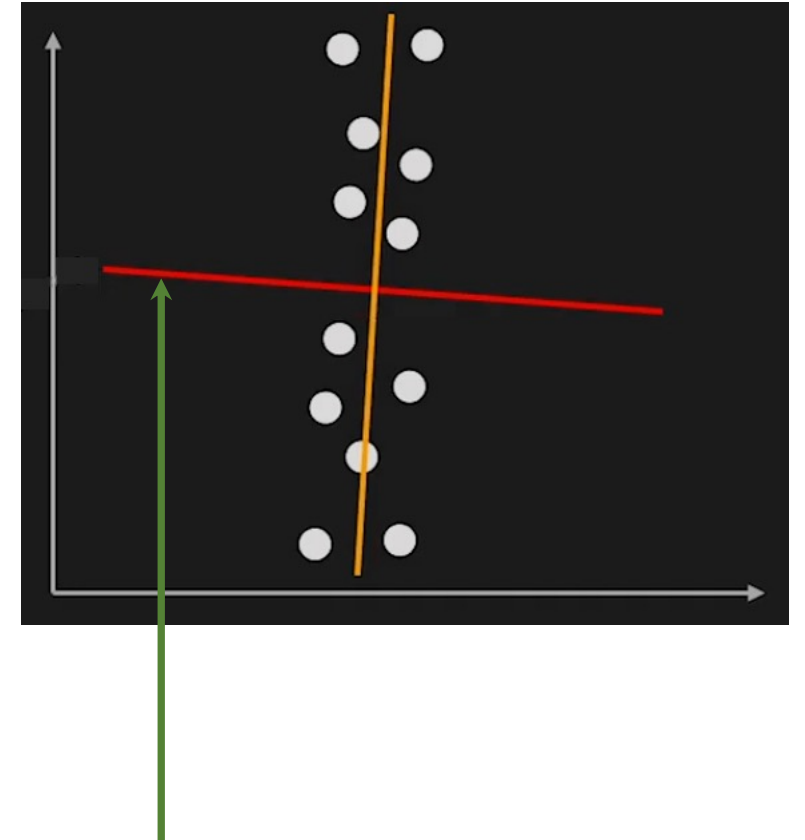
The solution: least square method

The problem:

Vertical lines.

An alternative solution:

Minimizing the perpendicular distance.



The line that minimizes E

VI. Line/curve boundary detection

Fitting lines to edges

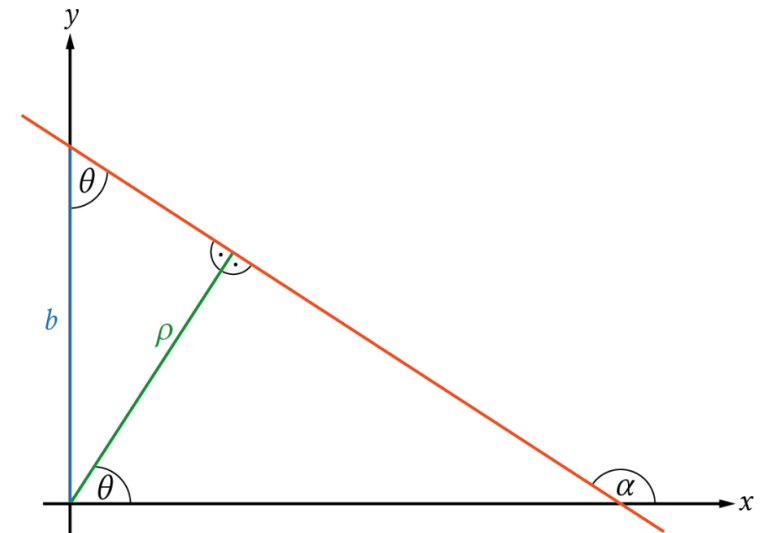
Minimizing the perpendicular distance

Switch to polar form:

$$x \cos \theta + y \sin \theta = \rho$$

- θ : angle between the x-axis and the line's normal vector
- ρ : distance from the origin to the line along the normal

$$y = \left(-\frac{\cos \theta}{\sin \theta} \right) x + \left(\frac{\rho}{\sin \theta} \right)$$



VI. Line/curve boundary detection

Fitting lines to edges

Minimizing the perpendicular distance

The **signed distance** from a point (x_i, y_i) to the line is:

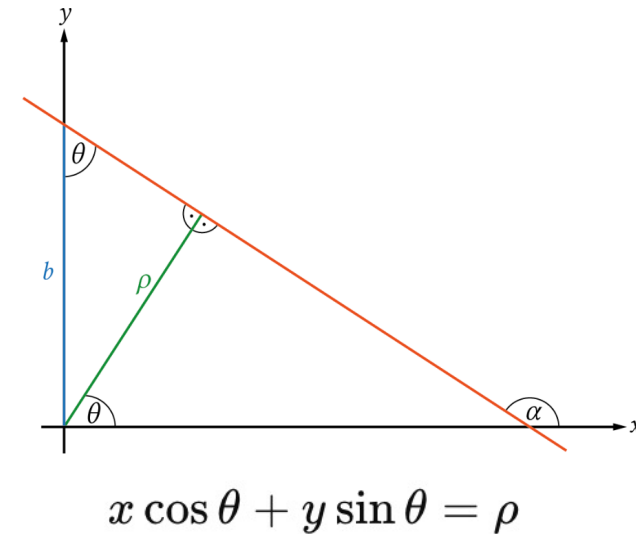
$$d_i = x_i \cos \theta + y_i \sin \theta - \rho$$

⇒ Minimize the **sum of squared distances**:

$$E(\theta, \rho) = \sum_{i=1}^n (x_i \cos \theta + y_i \sin \theta - \rho)^2$$

⇒ Solution:

- Perform a 2D PCA over the points (after centering them)
- The normal direction of the line is the eigenvector of the **smallest eigenvalue of the** covariance matrix from the centered data



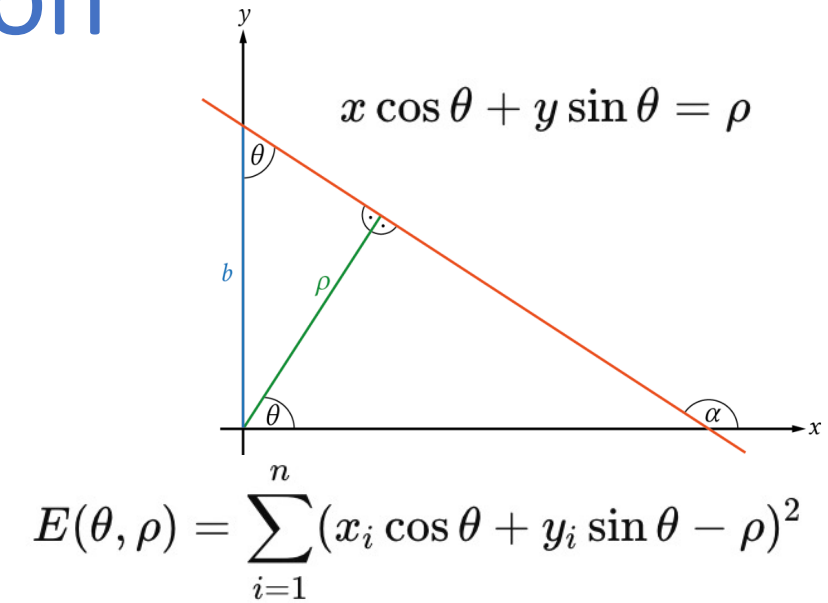
VI. Line/curve boundary detection

Fitting lines to edges

Minimizing the perpendicular distance

⇒ Solution: 2D PCA

1. Compute the mean $(\bar{x}, \bar{y})$
2. Center the data: $x'_i = x_i - \bar{x}$, $y'_i = y_i - \bar{y}$
3. Form the 2x2 covariance matrix from the centered data
4. Compute eigenvectors/values — pick the eigenvector of the **smallest eigenvalue** as the normal direction
5. Compute $\rho = \bar{x} \cos \theta + \bar{y} \sin \theta$

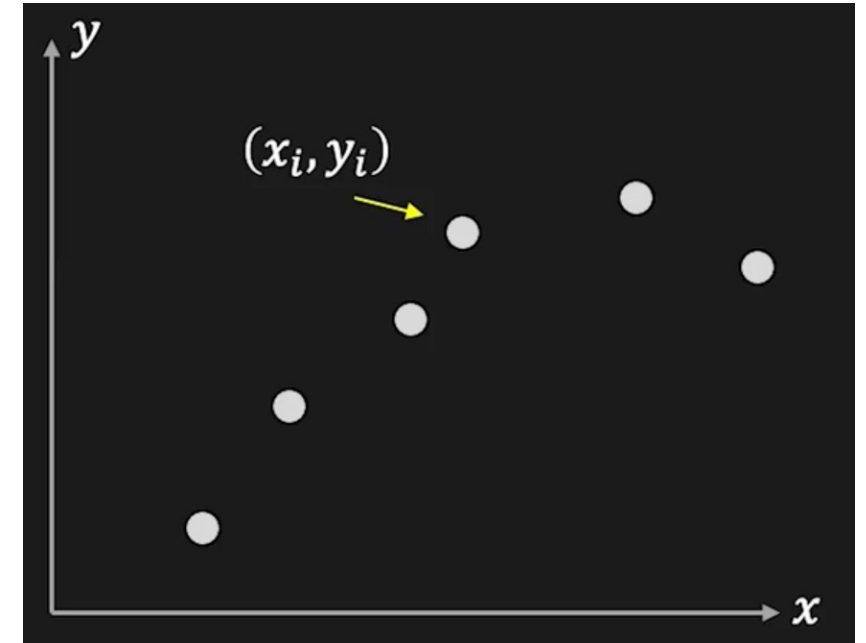


VI. Line/curve boundary detection

Fitting curves to edges

Edges: points (x_i, y_i)

Goal: find a polynomial that best fits the edge points.



VI. Line/curve boundary detection

Fitting curves to edges

Edges: points (x_i, y_i)

Goal: find a polynomial that best fits the edge points.

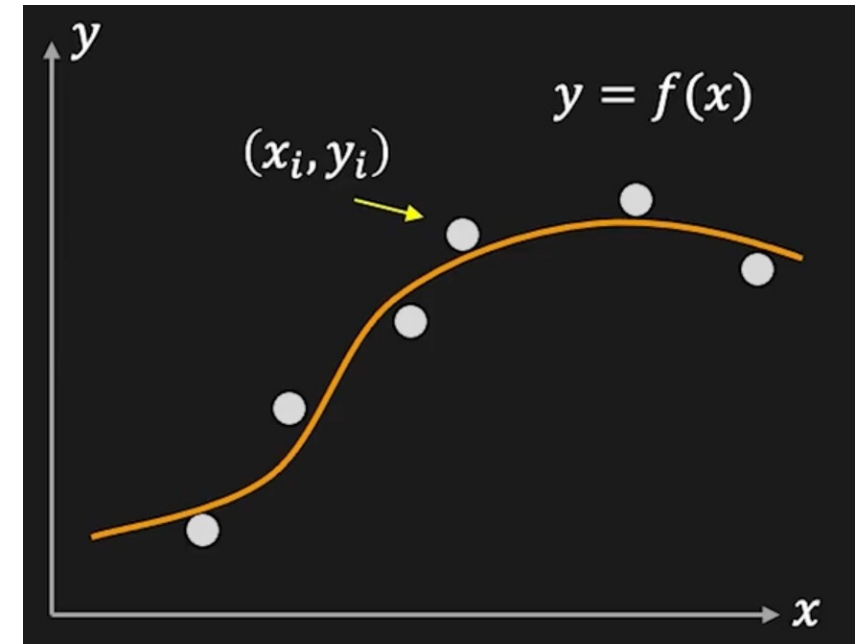
$$y = f(x) = ax^3 + bx^2 + cx + d$$

⇒ Minimize:

$$E = \frac{1}{N} \sum_i (y_i - ax_i^3 - bx_i^2 - cx_i - d)^2$$

⇒ Solve the linear system using the least square method:

$$\frac{\partial E}{\partial a} = 0; \quad \frac{\partial E}{\partial b} = 0; \quad \frac{\partial E}{\partial c} = 0; \quad \frac{\partial E}{\partial d} = 0;$$



VI. Line/curve boundary detection

Fitting curves to edges

Edges: points (x_i, y_i)

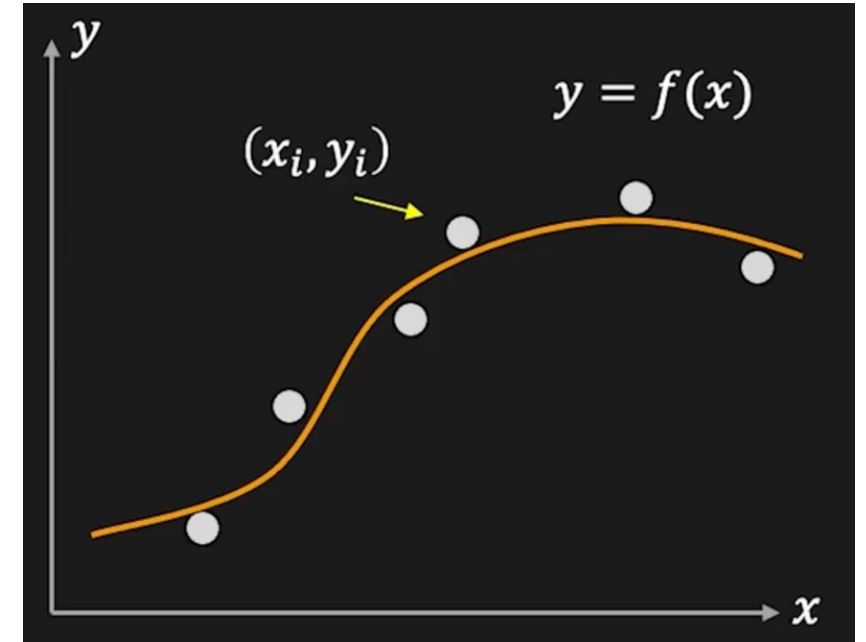
Goal: find a polynomial that best fits the edge points with a more general solution

$$y_0 = ax_0^3 + bx_0^2 + cx_0 + d$$

$$y_1 = ax_1^3 + bx_1^2 + cx_1 + d$$

...

$$y_n = ax_n^3 + bx_n^2 + cx_n + d$$



VI. Line/curve boundary detection

Fitting curves to edges

Edges: points (x_i, y_i)

Goal: find a polynomial that best fits the edge points with a more general solution

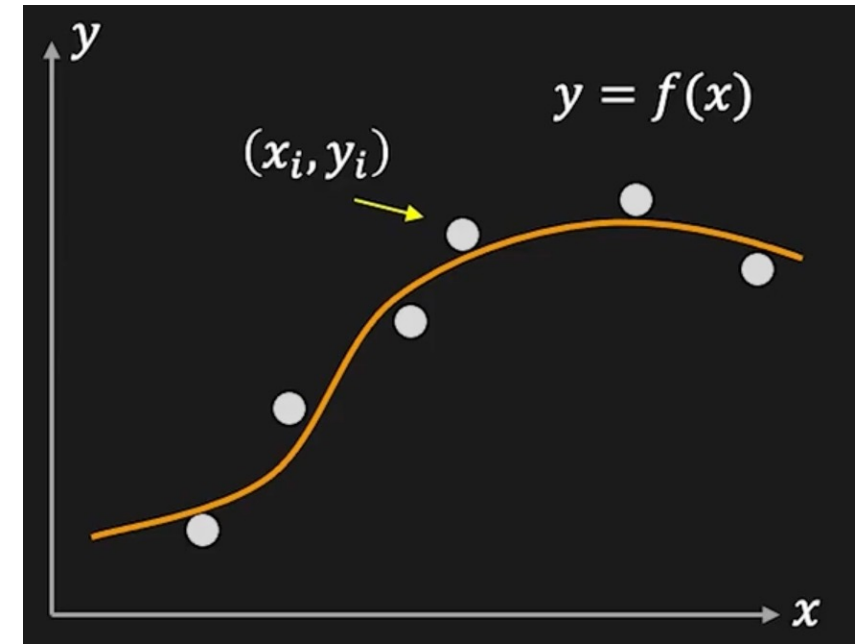
$$y_0 = ax_0^3 + bx_0^2 + cx_0 + d$$

$$y_1 = ax_1^3 + bx_1^2 + cx_1 + d$$

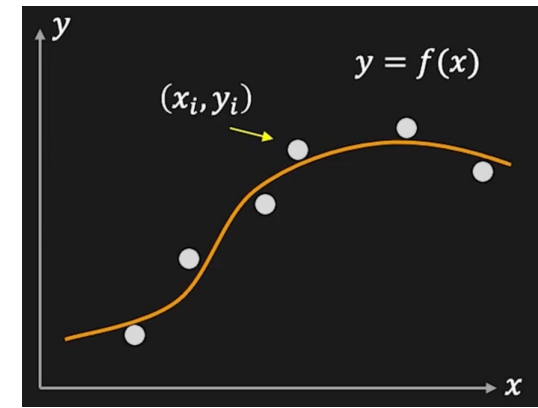
...

$$y_n = ax_n^3 + bx_n^2 + cx_n + d$$

⇒ Given many (x_i, y_i) , this is an over-determined linear system with 4 unknowns (a, b, c, d) .



VI. Line/curve boundary detection



⇒ Solving a linear system

An over-determined linear system with m unknowns $\{a_j\}$ where $j = 0 \dots m$ and n observations $\{(x_{i0}, \dots, x_{im}, y_i)\}$ where $i = 0 \dots n$ can be written in a matrix form:

$$\begin{bmatrix} x_{00} & x_{01} & \dots & x_{0m} \\ x_{10} & x_{11} & \dots & x_{1m} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n0} & x_{n1} & \dots & x_{nm} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix} \Rightarrow Xa = y$$

$X_{n \times m}$: Known

$a_{m \times 1}$: Unknown

$y_{n \times 1}$: Known

⇒ $X_{n \times m}$ is not necessarily a square matrix, thus it cannot be inverted.

VI. Line/curve boundary detection

⇒ Solving a linear system

We can use the least squares solution:

$$X\mathbf{a} = \mathbf{y}$$

$$X^T X \mathbf{a} = X^T \mathbf{y}$$

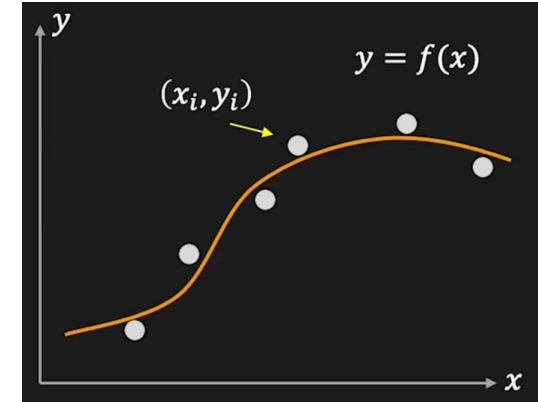
$X^T X$ is a $m \times m$ square matrix

$$(X^T X)^{-1} X^T X \mathbf{a} = (X^T X)^{-1} X^T \mathbf{y}$$

$$\mathbf{a} = (X^T X)^{-1} X^T \mathbf{y}$$

$$X^+ = (X^T X)^{-1} X^T$$

(Pseudo inverse)



VI. Line/curve boundary detection

The main problem with boundary detection:

How to know which edges in an image actually correspond to the boundary we are looking for.

=> One solution: **The Hough transform**

Ref.: Hough, P. V. C. Method and means for recognizing complex patterns, U.S. Patent 3,069,654, Dec. 18, 1962.